

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №5

з дисципліни «Аналіз та рефакторинг коду»

на тему «Розгортання програмної системи та демонстрація її роботи»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н.І.

Прийняв:

асистент каф. ПІ

Дашенков Д.С.

Харків 2025

5.1 Мета роботи

Метою роботи є набуття практичних навичок розгортання розробленої програмної системи у хмарному середовищі. Завдання включає перенесення серверної частини на публічний хостинг, налаштування хмарної бази даних, конфігурацію змінних середовища для безпечного доступу та перевірку працездатності інтегрованої системи (Backend + Database + IoT Client).

5.2 Архітектура розгортання

Для реалізації хмарної інфраструктури системи «Aquila» було обрано такі сервіси:

- Backend Hosting: Платформа Render. Використовується для хостингу Node.js додатку. Забезпечує безперервну інтеграцію (CI/CD) — автоматичну збірку та розгортання при оновленні коду в GitHub.
- Database: Сервіс Neon (Serverless PostgreSQL). Хмарна реляційна база даних, що забезпечує масштабованість, надійне зберігання даних та підтримку SSL-з'єднань.
- Version Control: GitHub. Виступає джерелом коду для автоматичного деплою.
- IoT Client – Мікроконтролер ESP32 (емульований у VS Code), що підключається до бекенду через HTTPS протокол.

5.3 Розгортання серверної частини (Backend)

Процес розгортання розпочався з підключення репозиторію GitHub до платформи Render. Було обрано репозиторій `rakomsakom/project`, який містить актуальний вихідний код серверної частини.

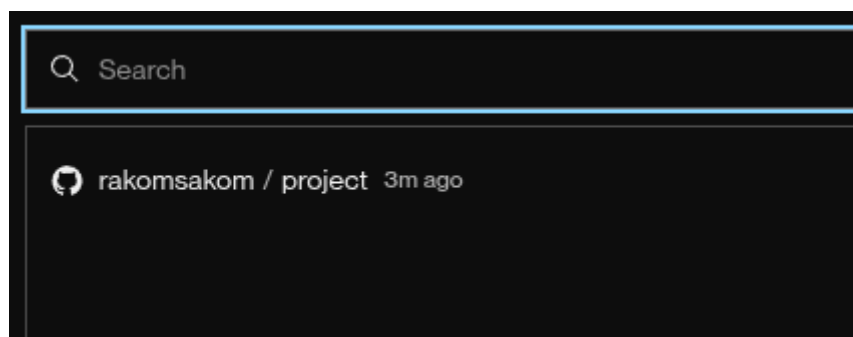


Рисунок 5.1 – Підключення репозиторію GitHub до Render

На етапі конфігурації веб-сервісу було задано параметри середовища виконання (Runtime: Node), команду збірки (npm install) та команду запуску (npm start). Також було налаштовано змінні середовища (Environment Variables), зокрема DATABASE_URL, для забезпечення безпечного з'єднання з базою даних Neon без збереження паролів у відкритому коді.

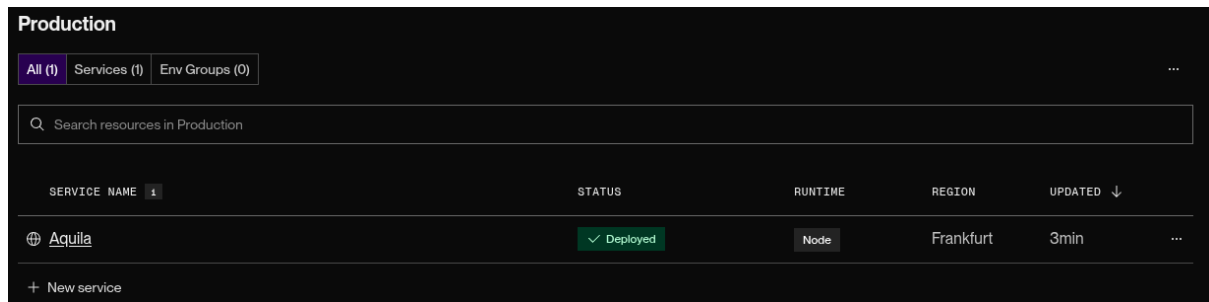


Рисунок 5.2 – Панель керування сервісом на Render

Після запуску процесу деплою платформа автоматично виконала встановлення залежностей та запуск серверу. У логах подій зафіксовано успішний старт додатку («Your service is live») та відкриття порту 10000 для вхідних з'єднань. Це підтверджує, що серверна частина коректно ініціалізована у хмарному середовищі.

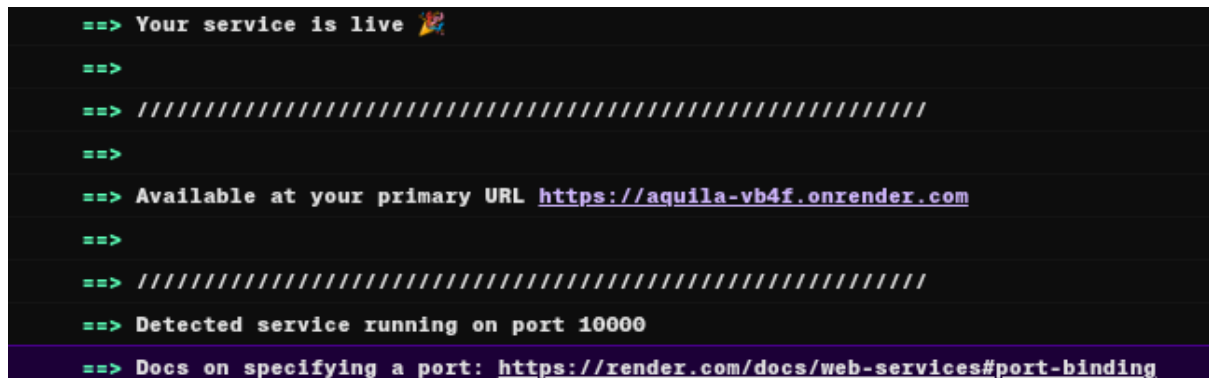


Рисунок 5.3 – Логи успішного запуску сервера

5.4 Налаштування бази даних та тестування

Паралельно було налаштовано хмарну базу даних Neon. Панель моніторингу відображає активність використання обчислювальних ресурсів та пам'яті, що свідчить про активне підключення серверної частини та виконання SQL-запитів.

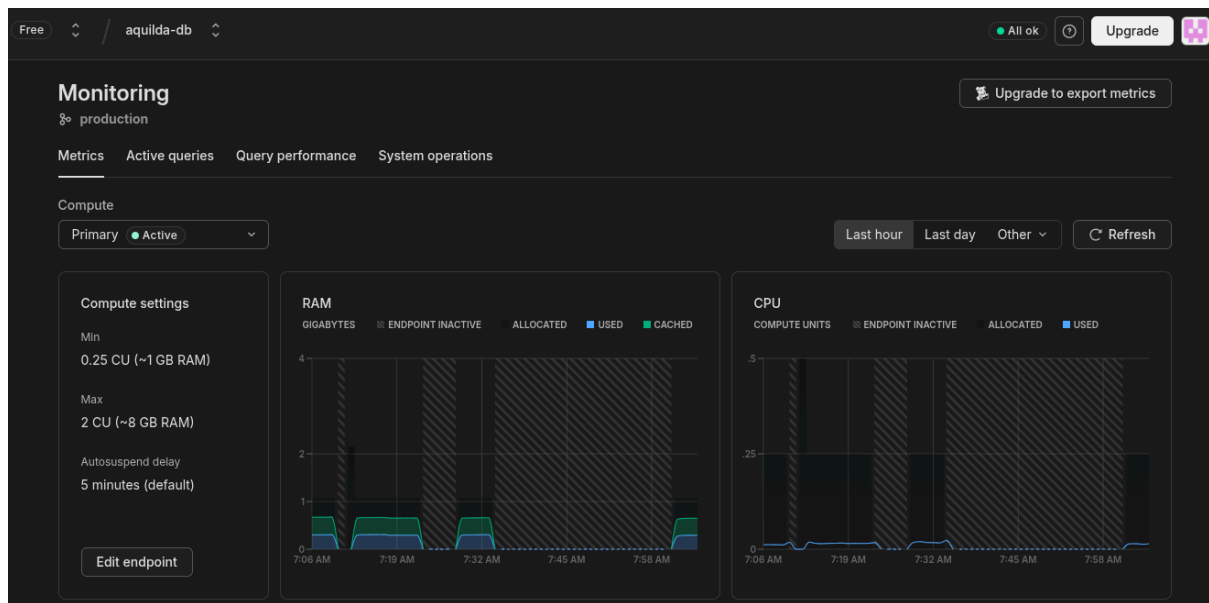


Рисунок 5.4 – Моніторинг активності бази даних Neon

Фінальним етапом стала перевірка працездатності всієї системи. За допомогою клієнтського запиту було отримано дані з API розгорнутого серверу. Відповідь у форматі JSON містить інформацію про пристрій (device_id), а також телеметричні дані (температура, вологість, рівень світла та шуму), які були передані IoT-клієнтом та збережені у базі даних.

```
[
  {
    "id": "1",
    "device_id": "cfee9aa2-d864-419a-ae2b-6b89a3005c68",
    "temperature": 25.5,
    "humidity": 60,
    "light_level": 500,
    "sound_level": 45,
    "recorded_at": "2025-12-22T15:35:26.209Z"
  }
]
```

Рисунок 5.5 – Результат запиту до хмарного API (отримання телеметрії)

Отримані дані ("temperature": 25.5, "light_level": 500) відповідають значенням, що генеруються емулятором IoT-пристрою, що підтверджує коректність роботи повного циклу передачі даних через інтернет.

5.5 Висновки

У ході лабораторної роботи було успішно виконано розгортання програмної системи моніторингу «Aquila». Реалізовано хмарну

архітектуру: Серверну частину перенесено на платформу Render, а базу даних — на NeonDB. Забезпечено автоматичне оновлення серверу при змінах у репозиторії GitHub. Конфіденційні дані (доступи до БД) винесено у змінні середовища хмарного хостингу. Підтверджено, що IoT-клієнт успішно передає дані на віддалений сервер, вони коректно зберігаються в базі даних та доступні через публічний API. Система готова до експлуатації у реальних умовах.

ДОДАТОК А

Слайди презентації



Рисунок А.1 – Титульний слайд



Рисунок А.2 – Вступ

Архітектура Backend'у

Серцем системи є сервер RESTful API, заснований на **Node.js**, що виступає посередником між пристроями та користувачем. Використано складну реляційну базу даних **PostgreSQL** з ідентифікаторами UUID для масштабованості та безпеки.

Структура бази даних підтримує логічне **зонування приміщень**, дозволяючи керувати групами пристроїв як єдиним цілим.

Рисунок А.3 – Бекенд проєкту

Цілісність Даних

Для забезпечення надійності даних, впроваджено механізм **ACID-транзакцій** для видалення пов'язаних сутностей (наприклад, кімнати з датчиками та історією). Це гарантує цілісність бази даних навіть у випадку збою.

Рисунок А.4 – Дані проєкту

Бізнес-логіка та Адміністрування

Придушення сигналів тривоги

Алгоритм блокує дублювання сповіщень про ту саму проблему протягом певного часу, запобігаючи "спаму".

Модель доступу RBAC

Система адміністрування включає модель доступу на основі ролей для контролю прав користувачів.

Журнали аудиту

Реєструють дії користувачів, забезпечуючи повну прозорість та можливість відстеження.

Експорт телеметрії

Функціонал експорту телеметричних даних для подальшого аналізу та звітності.

Рисунок А.5 – Бізнес-логіка проєкту

Клієнт IoT: ESP32

Вбудоване програмне забезпечення для мікроконтролера **ESP32** збирає дані з датчиків: **DHT22** (температура/ вологість), **LDR** (освітлення), **PIR** (рух) та емулятор мікрофона.

Ключові особливості — це **Edge Computing** та **стійкість до збоїв**.

Рисунок А.6 – IoT клієнт

Периферійні обчислення та відмовостійкість

1 Обробка даних на пристрої

Пристрій самостійно обробляє дані та перевіряє критичні пороги.

2 Емуляція Камери

Після виявлення руху система імітує роботу камери, надсилаючи посилання на знімок.

3 Офлайн-буферизація

У разі втрати Інтернет-з'єднання дані буферизуються та синхронізуються після його відновлення.

Рисунок А.7 – Особливості IoT клієнту

Розгортання в хмарі

Проєкт був перенесений у хмарне середовище, ставши повноцінним продуктом SaaS. Сервер працює на **Render** з CI/CD, а база даних на **Neon (Serverless PostgreSQL)**. Комунікація захищена протоколом HTTPS.

Рисунок А.8 – Розгортання проєкту

Надійний Контроль 24/7

Система Aquila забезпечує надійний контроль стану приміщень цілодобово, 7 днів на тиждень, пропонуючи душевний спокій та ефективність.

Рисунок А.10 – Завершальні слова